

Software Engineering (Project A)



NightOUT

Authors

Riccardo Masarin	10708900
Manuel Mannucci	10761728
Alessandro Forlani	10753247

Requirements Analysis and Specification Document



POLITECNICO
MILANO 1863

MSc Automation and Control Engineering
July 24, 2026

Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	1
1.3	Definitions, Acronyms, Abbreviations	1
1.4	Document Structure	2
2	Overall Description	2
2.1	Product Perspective	2
2.2	Scenarios	2
2.3	Domain Model	4
2.4	User Characteristics	5
2.5	Product Functions	6
2.6	Functional Requirements	7
2.6.1	User and Profile Requirements	7
2.6.2	Event Discovery Requirements	7
2.6.3	Recommendation Requirements	8
2.6.4	Reservation and Ticketing Requirements	8
2.6.5	Pregame Requirements	9
2.6.6	Social Layer Requirements	10
2.6.7	Notification Requirements	10
2.6.8	Venue Manager Requirements	10
2.7	Nonfunctional Requirements	11
2.7.1	Usability	11
2.7.2	Performance	11
2.7.3	Reliability	11
2.7.4	Robustness	12
2.7.5	Maintainability	12
2.7.6	Security and Privacy	12
2.8	Assumptions, Dependencies and Constraints	12
2.8.1	Domain Assumptions	12
2.8.2	Dependencies	13
2.8.3	Constraints	13
3	Additional Models	14
3.1	Requirements-Level Sequence Diagrams	14
3.1.1	User Reserves an Event	14
3.1.2	User Joins a Pregame Room	15
3.1.3	Venue Manager Creates an Event	15
3.1.4	Waiting List Promotion	16
3.2	Finite State Machines	17
3.2.1	Reservation Lifecycle FSM	17
3.2.2	Pregame Room Capacity State Machine	18

1 Introduction

1.1 Purpose

The goal of this project is to design a software platform that combines event discovery, pregame organization, digital ticketing, venues and PR support. Users can discover clubs and bars, coordinate with friends or verified groups, access tickets and receive clearer information before choosing where to go. At the same time, clubs, bars and PRs can use NightOUT as a visibility, promotion and customer management tool.

1.2 Scope

NightOUT is intended as a web application for nightlife discovery and organization. The system allows users to browse events, filter them according to relevant criteria, reserve spots, join pregame rooms, interact with events information, and receive notifications related to friends and reservations. The system also supports venue managers and PRs, who can manage venues, publish events, approve or define promotions, and access basic statistics related to their events. The prototype focuses on the following macro-features:

1. Event discovery and recommendation

Users can browse nightlife events and receive suggested events based on music preferences, saved events, social proximity, geographical distance, and booking history.

2. Event search and filtering

Users can filter events by date, location, music genre, venue category, and price or entry condition.

3. Event popularity dashboard

Users can browse events ordered by popularity metrics such as most saved events, most booked events, music genre popularity, and similar events attended.

4. Social network layer

Users can have personal profiles, follow or connect with other users, see participation information, and receive suggestions based on shared interests.

5. Pregame section

Users can create or join temporary groups before an event, either in a user-defined location or in a bar available as a pregame venue.

6. Reservation and ticketing module

Users can reserve a spot for an event, join a waiting list, cancel a reservation, and view the reservation status in their profile. The prototype does not manage real payments.

7. Venue manager features

Venue managers can publish events, manage event information, approve promotions, apply changes, and access basic statistics.

1.3 Definitions, Acronyms, Abbreviations

User: registered person who uses NightOUT to discover events, save events, reserve spots, join pregame rooms, and interact with social features.

Venue: physical place such as a club, bar, lounge bar, or live music venue where nightlife events take place.

PR: club promoters that share promotions codes to increase the users affluence to an event.

Venue Manager: verified account responsible for managing one or more venues and publishing events or promotions.

Event: nightlife activity associated with a venue. It includes information such as title, venue, date and time, music genre, dress code, price or entry condition, capacity, and possible promotions.

Reservation: digital record representing a user request to attend an event. A reservation may be pending, confirmed, cancelled, expired, or placed in the waiting list.

Waiting List: list of users who requested a reservation for a full event and may be promoted if a spot becomes available.

Pregame Room: temporary group created before an event, allowing users to coordinate a meeting before attending the event.

Promotion: an offer, discount, or special entry condition associated with a venue or event.

1.4 Document Structure

This document is organized as follows:

Section 1 introduces the purpose, scope, definitions, and structure of the document.

Section 2 provides the overall description of the system, including product perspective, scenarios, domain model, user characteristics, product functions, functional requirements, nonfunctional requirements, assumptions, dependencies, and constraints.

Section 3 presents additional requirement-level models, including sequence diagrams, finite state machines, and selected logical specifications.

2 Overall Description

2.1 Product Perspective

The system is developed as a web application supported by a Java Spring Boot backend and a relational database. The frontend is optimized for smartphone usage, but it is implemented as a web application to reduce prototype complexity and focus the development effort on the backend architecture, domain model, and business logic.

NightOUT does not interact with real payment systems or external ticketing providers in the prototype version. Instead the system simulates the reservation and ticketing process by storing reservation records and showing digital reservation confirmations.

The prototype uses a realistic synthetic dataset generated by the development team. This dataset includes users, venue managers, venues, events, reservations, social relations, pregame rooms, promotions, and notifications. This choice allows the system to demonstrate all relevant scenarios without depending on external APIs or real commercial data.

2.2 Scenarios

The following scenarios describe relevant interactions between the system and the external world. They are written at requirement level and focus on common shared phenomena between users, venue managers, events, reservations, and the NightOUT system.

Scenario 1: Discovering an Event

A user opens NightOUT to decide where to go during the evening. The system displays a feed of suggested nightlife events. The user filters events by date, location, music genre, venue category, and price or entry condition. The user selects one event and views the event page,

which includes venue name, location, date and time, music genre, age target, dress code, price or entry conditions, available promotions, and popularity information.

Relevant actors: User

Relevant system functions: event browsing, filtering, event detail visualization, popularity information.

Scenario 2: Reserving a Spot for an Event

A registered user selects an event and requests a reservation, creating a ticket with pending state. If the event has available capacity, the system changes the state of the ticket to confirmed and displays it in the user profile or ticket section. If the event is full, the system places the user in the waiting list and shows the corresponding reservation status.

Relevant actors: User

Relevant system functions: reservation creation, capacity check, reservation status management, waiting list management.

Scenario 3: Waiting List notification

A user is in the waiting list for a full event. Another user cancels a confirmed reservation. The system detects that a spot is available and promotes the first eligible waiting list reservation to confirmed. The promoted user receives a notification.

Relevant actors: User

Relevant system functions: reservation cancellation, waiting list update, notification.

Scenario 4: Joining a Pregame Room

A user opens an event page and browses the pregame rooms associated with that event. The user selects a room with available capacity and joins it. The system updates the room participant list and displays the updated room information. If the room is full, the system prevents the user from joining it.

Relevant actors: User

Relevant system functions: pregame room browsing, join/leave room, participant management.

Scenario 5: Creating a Pregame Room

A registered user creates a pregame room linked to an event. The user specifies a meeting location, meeting time, maximum number of participants, and short description. The system stores the room and makes it visible to other users browsing pregame options for the same event.

Relevant actors: User

Relevant system functions: pregame room creation, event-room association.

Scenario 6: Venue Manager Publishes an Event

A verified venue manager logs into NightOUT and creates a new event for one of their venues. The manager provides event title, date and time, venue, music genre, dress code, price or entry condition, capacity, and possible promotions. The system stores the event and makes it available in the event feed of the users.

Relevant actors: Venue Manager

Relevant system functions: event creation, venue management, promotion management.

Scenario 7: Social Event Discovery

A user opens the app and views events that are popular among friends or followed users. The

system uses social relations and event participation information to highlight events attended or saved by people connected to the user.

Relevant actors: User

Relevant system functions: social layer, event participation, recommendation, popularity computation.

Scenario 8: Event Notification

A user receives a notification when a relevant event occurs, such as a friend joining an event, a reservation changing status, a waiting list spot becoming available, or an event being updated or cancelled.

Relevant actors: User

Relevant system functions: notification management, event updates, reservation status updates, social updates.

2.3 Domain Model

The main domain entities are:

- User
- Venue Manager
- Venue
- Event
- Reservation
- Pregame Room
- Promotion
- Notification
- Social Relation
- Event Participation
- Music Genre
- Venue Category
- Location

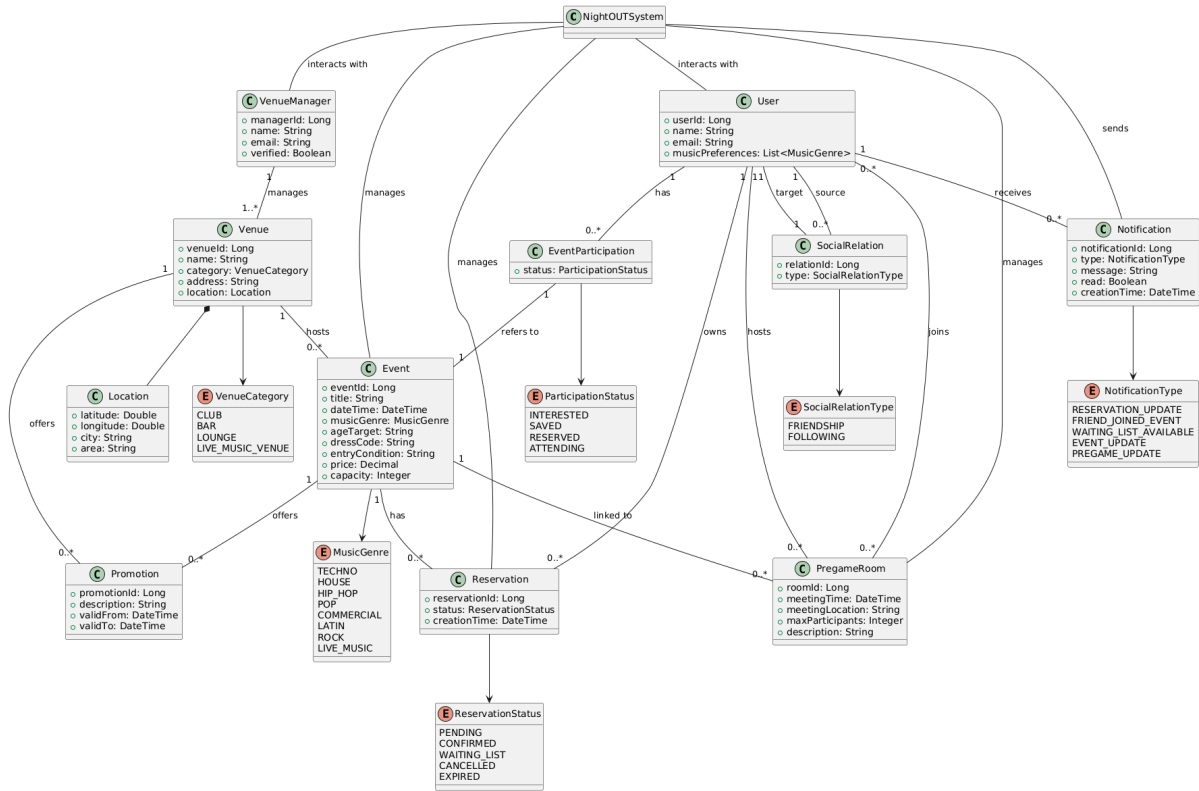


Figure 1: Domain Model

2.4 User Characteristics

NightOUT has three main categories of users.

1. Regular Users

Regular users are mainly students, young adults, and nightlife participants who want to discover events, understand what is happening in the city, coordinate with friends, reserve spots, and join pregame groups. They are expected to use the system mainly from mobile devices. Therefore, usability, clarity of event information, and fast access to the main actions are important aspects. Regular users need to:

- browse and filter events;
- view event details;
- save events;
- reserve spots or join waiting lists;
- view reservation status;
- join or create pregame rooms;
- see friends participation information;
- receive notifications.

2. Venue Managers

Venue managers are verified users responsible for one or more venues. They use the system to publish and manage events, define promotions, update event information, and access basic statistics. Venue managers need to:

- create events;
- update event information;
- define capacity and entry conditions;
- manage promotions;
- view basic statistics such as saved events, reservations, and popularity indicators.

3. PRs

PRs are verified users responsible for events promotions. They own their personal Promo code to promote discounts on tickets that they sold.

2.5 Product Functions

1. Event Discovery

The system allows users to browse a feed of nightlife events.

2. Event Filtering

The system allows users to filter events by date, location, music genre, venue category, and price or entry condition.

3. Event Details

The system allows users to view detailed information about an event, including venue, location, date and time, music genre, age target, dress code, price, entry conditions, promotions, and popularity information.

4. Event Recommendation

The system suggests events based on user preferences, saved events, common friends, geographic distance, and booking history.

5. Event Popularity Dashboard

The system provides an ordered view of events based on popularity metrics such as most saved, most booked, friends attending, and genre trends.

6. Reservation and Ticketing

The system allows users to reserve spots, join waiting lists, cancel reservations, and view reservation status.

7. Pregame Room Management

The system allows users to create, search, join, and leave pregame rooms linked to events.

8. Social Layer

The system allows users to create friendship relations and view event participation information from friends or followed users.

9. Notification Management

The system notifies users about reservation state changes, waiting list availability, friends joining events, and event updates.

10. Venue Manager Event Management

The system allows venue managers to create, update, and manage events and promotions.

2.6 Functional Requirements

The following requirements use the optative form "shall", consistently with the requirements engineering convention.

2.6.1 User and Profile Requirements

1. User Login

The system shall allow a registered user to log into the system.

2. User Profile Visualization

The system shall allow a registered user to view their profile information.

3. User Preferences

The system shall allow a registered user to define music preferences.

4. Saved Events

The system shall allow a registered user to save an event.

5. View Saved Events

The system shall allow a registered user to view their saved events.

2.6.2 Event Discovery Requirements

1. Browse Events

The system shall allow users to browse available nightlife events.

2. View Event Details

The system shall allow users to view the details of a selected event.

3. Event Information Completeness

For each event, the system shall display at least title, venue name, location, date and time, music genre, dress code, price or entry condition, and capacity information.

4. Filter Events

The system shall allow users to filter events by date, location, music genre, venue category, and price or entry condition.

Let **events** be the set of available events, **criteria** the selected filtering criteria, and **res** the set of returned events.

$$\text{events} \neq \text{NIL}$$

$$\forall e (e \in \text{res} \implies e \in \text{events} \wedge \text{satisfies}(e, \text{criteria}))$$

This means that every returned event must belong to the original event set and must satisfy the selected criteria.

5. Empty Filter Result

When no event satisfies the selected filters, the system shall display an empty result message.

6. Event Popularity View

The system shall allow users to view events ordered according to selected popularity criteria.

2.6.3 Recommendation Requirements

1. Recommended Event Feed

The system shall provide a suggested event feed for registered users.

2. Music-Based Recommendation

The system shall consider user music preferences when generating event recommendations.

3. Social-Based Recommendation

The system shall consider friends event participation when generating event recommendations.

4. Distance-Based Recommendation

The system shall consider geographic distance when generating event recommendations.

2.6.4 Reservation and Ticketing Requirements

1. Request Reservation

The system shall allow a registered user to request a reservation for an event.

2. Confirm Reservation with Available Capacity

When an event has available capacity, the system shall create a confirmed reservation for the requesting user.

```
user.isRegistered == true ∧ event.capacity > event.confirmedReservations
⇒ ∃r (
    r.user = user ∧
    r.event = event ∧
    r.status = CONFIRMED
)
```

3. Waiting List for Full Events

When an event has no available capacity, the system shall place the requesting user in the waiting list.

```
user.isRegistered == true ∧ event.capacity == event.confirmedReservations
⇒ ∃r (
    r.user = user ∧
    r.event = event ∧
    r.status = WAITING_LIST
)
```

4. View Reservation Status

The system shall allow a registered user to view the status of their reservations.

5. Cancel Reservation

The system shall allow a registered user to cancel their reservation.

6. Promote Waiting List Reservation

When a confirmed reservation is cancelled and a waiting list exists for the event, the system shall promote an eligible waiting list reservation to confirmed.

7. Prevent Duplicate Active Reservations

The system shall not allow a user to have more than one active reservation for the same event.

```

active(r)  $\iff$ 
(r.status = CONFIRMED  $\vee$ 
 r.status = PENDING  $\vee$ 
 r.status = WAITING_LIST)

 $\forall r_1, r_2$  (
  active(r1)  $\wedge$  active(r2)  $\wedge$ 
  r1.user == r2.user  $\wedge$ 
  r1.event == r2.event
 $\implies$  r1 = r2)

```

8. Reservation Status Values

The system shall represent each reservation using one of the following states: pending, confirmed, waiting list, cancelled, or expired.

9. Digital Reservation Confirmation

The system shall display a digital reservation confirmation for confirmed reservations.

2.6.5 Pregame Requirements

1. Create Pregame Room

The system shall allow a registered user to create a pregame room linked to an existing event.

2. Pregame Room Information

For each pregame room, the system shall store host, linked event, meeting location, meeting time, maximum number of participants, current participants, and description.

3. Browse Pregame Rooms

The system shall allow users to browse pregame rooms linked to a selected event.

4. Join Pregame Room

The system shall allow a registered user to join a pregame room if the room has available capacity.

```

user.isRegistered == true  $\wedge$ 
room.currentParticipants < room.maxParticipants  $\wedge$ 
room.linkedEvent  $\neq$  NIL
 $\implies$  user  $\in$  room.participants

```

5. Prevent Joining Full Room

The system shall prevent a user from joining a pregame room whose maximum capacity has been reached.

6. Leave Pregame Room

The system shall allow a registered user to leave a pregame room previously joined.

7. Prevent Duplicate Participation

The system shall not allow the same user to join the same pregame room more than once.

$$\forall u, r \text{ (countOccurrences}(u, r.\text{participants}) \leq 1)$$

2.6.6 Social Layer Requirements

1. Friends Relation Creation

The system shall allow a registered user to create a social relation with another registered user through a friendship mechanism.

2. View Friends

The system shall allow a registered user to view their social connections.

3. Event Participation Status

The system shall allow a registered user to indicate interest in or participation in an event.

4. Friends Attending Event

The system shall allow a registered user to view whether social connections are attending or interested in an event.

5. Activity Feed

The system shall provide a simple activity feed showing relevant event-related activities among the user's social connections.

2.6.7 Notification Requirements

1. Reservation Notification

The system shall notify a user when one of their reservations changes status.

2. Waiting List Notification

The system shall notify a user when their waiting list reservation is promoted to confirmed.

3. Friend Event Notification

The system shall notify a user when a social connection joins or saves an event.

4. Event Update Notification

The system shall notify users affected by relevant event changes, such as event cancellation or important event information updates.

5. View Notifications

The system shall allow registered users to view their notifications.

6. Mark Notification as Read

The system shall allow registered users to mark notifications as read.

2.6.8 Venue Manager Requirements

1. Venue Manager Login

The system shall allow a verified venue manager to log into the system.

2. Create Event

The system shall allow a verified venue manager to create an event for a venue they manage.

3. Update Event

The system shall allow a verified venue manager to update event information for events they manage.

4. Manage Event Capacity

The system shall allow a verified venue manager to define and update the capacity of an event.

5. Create Promotion

The system shall allow a verified venue manager to create promotions associated with their venues or events.

6. View Event Statistics

The system shall allow a verified venue manager to view basic statistics about their events, including number of saved events, number of reservations, and waiting list size.

2.7 Nonfunctional Requirements**2.7.1 Usability****1. Mobile-Oriented Interface**

The system shall provide a responsive user interface optimized for smartphone usage.

2. Event Reservation Simplicity

The system shall allow a user to request a reservation from the event detail page in no more than three main interaction steps.

3. Event Information Clarity

The system shall display the main event information in a clear and structured way.

2.7.2 Performance**1. Event Feed Response Time**

The system shall return the event feed within 2 seconds for the synthetic dataset used in the prototype.

2. Filter Response Time

The system shall return filtered event results within 2 seconds for the synthetic dataset used in the prototype.

3. Reservation Response Time

The system shall return a reservation status response within 2 seconds under normal prototype execution conditions.

2.7.3 Reliability**1. Reservation Consistency**

The system shall maintain consistency between event capacity and confirmed reservations.

2. No Overbooking

The system shall prevent the number of confirmed reservations for an event from exceeding the event capacity.

3. Waiting List Consistency

The system shall maintain a consistent waiting list order for full events.

2.7.4 Robustness

1. Invalid Reservation Request

The system shall handle invalid reservation requests without crashing.

2. Invalid Pregame Join Request

The system shall handle invalid pregame room join requests without crashing.

3. Empty Search Results

The system shall handle empty event search results by displaying an appropriate message.

2.7.5 Maintainability

1. Modular Backend Structure

The backend shall be organized into separate components for user management, event management, reservation management, pregame management, notification management, recommendation, and data access.

2. Separation of Concerns

The system shall separate presentation logic, application logic, domain logic, and persistence logic.

3. Pattern-Based Extensibility

The system design shall support the extension of reservation states, notification types, filtering strategies, popularity strategies, and recommendation strategies.

2.7.6 Security and Privacy

1. Authenticated Actions

The system shall require authentication for reservation, pregame participation, social interaction, and venue manager operations.

2. User Data Visibility

The system shall prevent non-authenticated users from accessing private user profile information.

3. Venue Manager Authorization

The system shall allow venue managers to modify only venues and events they manage.

2.8 Assumptions, Dependencies and Constraints

2.8.1 Domain Assumptions

1. Synthetic Dataset

The prototype uses a realistic synthetic dataset generated by the development team.

2. No Real-Time Commercial Data

The prototype does not depend on real-time data from real venues, ticketing platforms, or external event providers.

3. No Real Payments

The prototype does not process real payments.

4. Simulated Ticketing

Digital reservation confirmations are simulated and do not represent legally valid tickets.

5. Verified Venue Managers

Venue managers are assumed to be verified accounts in the prototype dataset.

6. User Social Relations

Social relations are assumed to be generated or manually inserted in the synthetic dataset.

2.8.2 Dependencies**1. Backend Framework**

The system depends on a Java Spring Boot backend.

2. Frontend Framework

The system depends on a React mobile-first web frontend.

3. Database

The system depends on a relational database storing users, events, reservations, pregame rooms, promotions, notifications, PR codes and friendship relations.

2.8.3 Constraints**1. Programming Language**

The backend shall be implemented in Java.

2. Backend Architecture

The backend shall expose REST APIs.

3. Frontend Type

The frontend shall be implemented as a responsive web application optimized for mobile usage.

4. No External Payment Integration

The prototype shall not integrate external payment systems.

5. No External Event API Requirement

The prototype shall not require external event APIs.

3 Additional Models

3.1 Requirements-Level Sequence Diagrams

3.1.1 User Reserves an Event

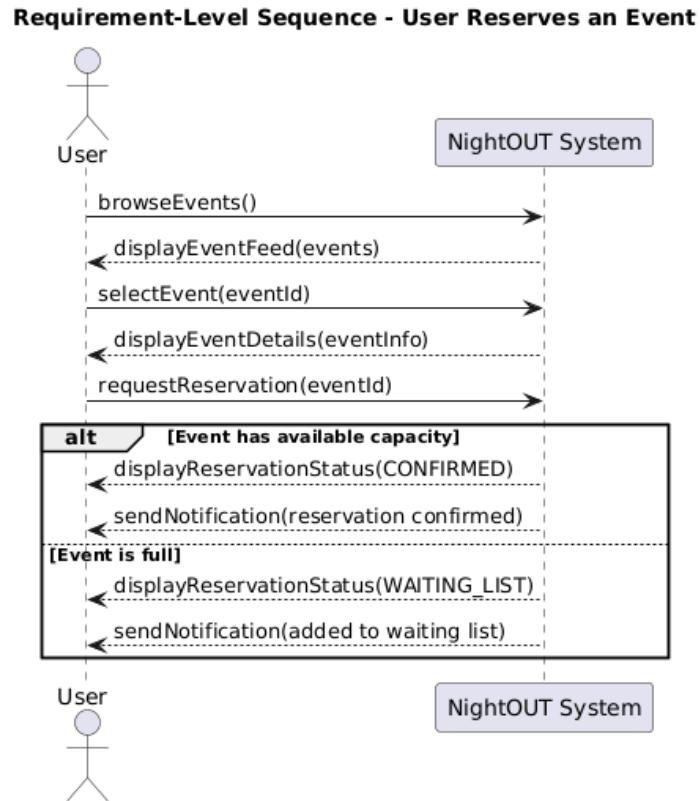


Figure 2: Requirement-Level Sequence - User Reserves an Event

3.1.2 User Joins a Pregame Room

Requirement-Level Sequence - User Joins a Pregame Room

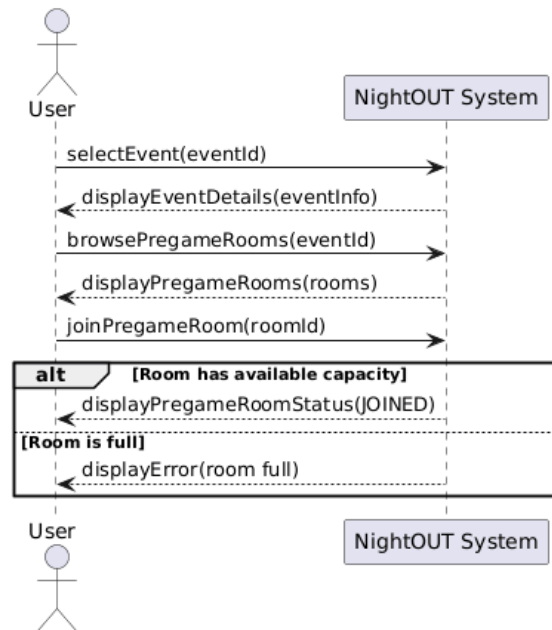


Figure 3: Requirement-Level Sequence - User Joins a Pregame Room

3.1.3 Venue Manager Creates an Event

Requirement-Level Sequence - Venue Manager Creates an Event

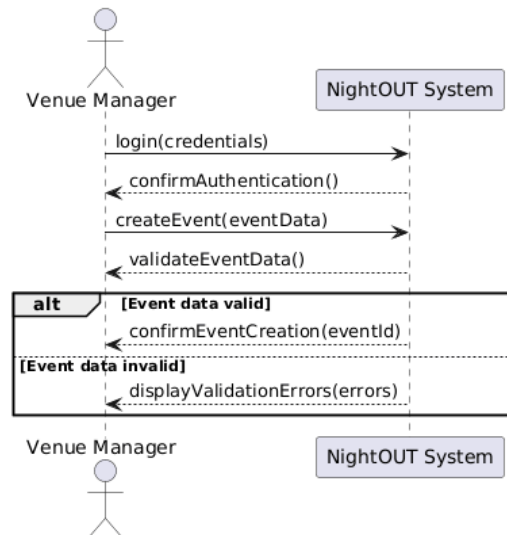


Figure 4: Requirement-Level Sequence - Venue Manager Creates an Event

3.1.4 Waiting List Promotion

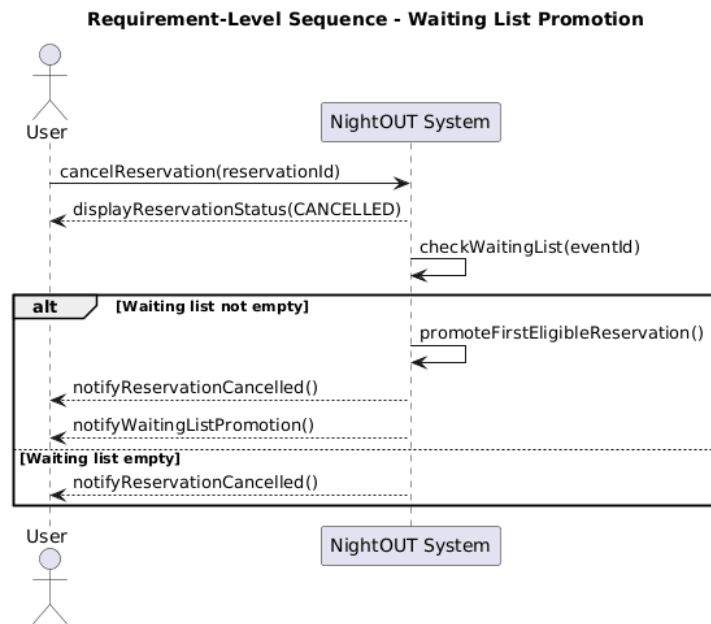


Figure 5: Requirement-Level Sequence - Waiting List Promotion

3.2 Finite State Machines

3.2.1 Reservation Lifecycle FSM

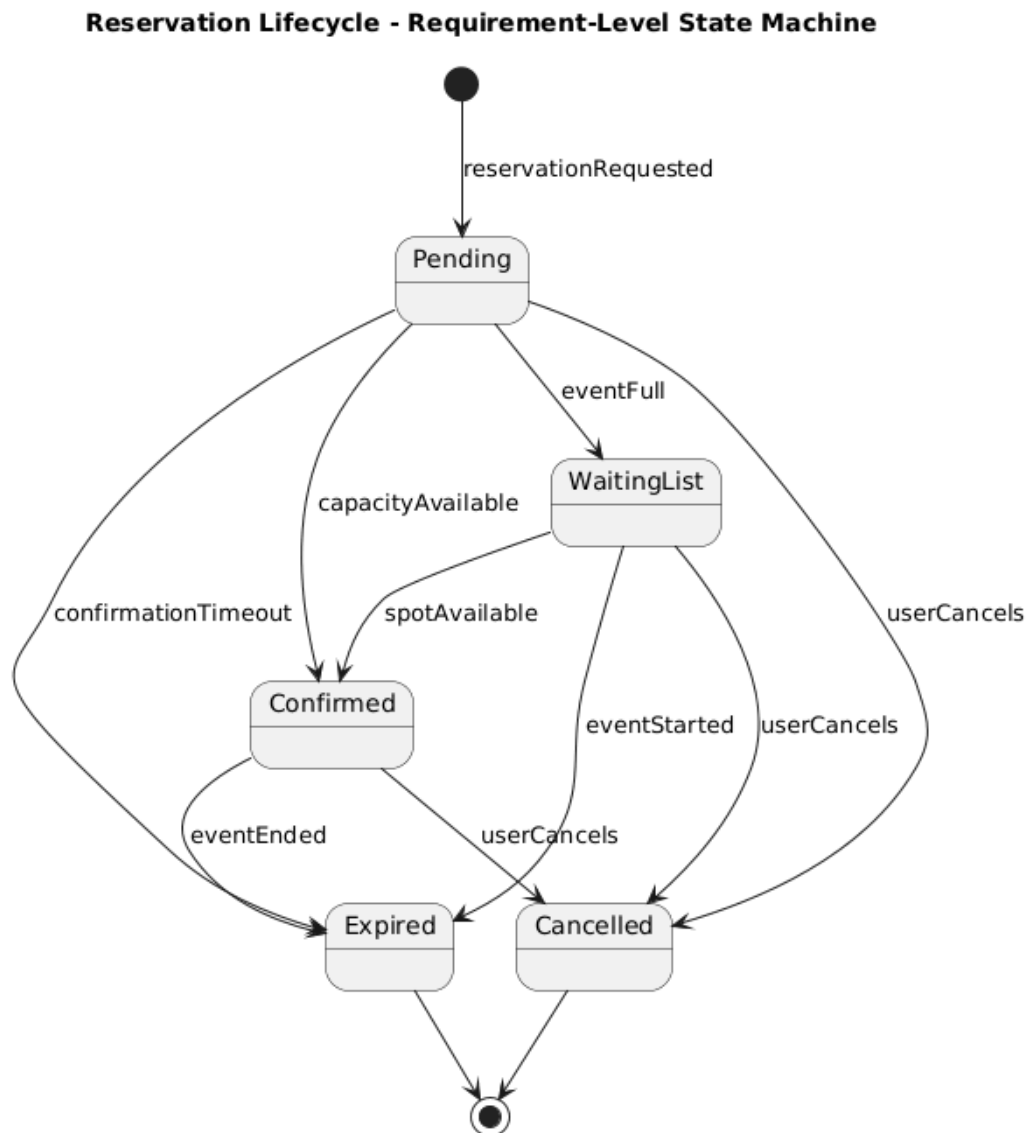


Figure 6: Reservation Lifecycle - Requirement-Level State Machine

A reservation is created when a user requests to attend an event. Initially, the reservation may be pending. If the event has available capacity, the reservation becomes confirmed. If the event is full, the reservation is placed in waiting list. A user may cancel a pending, waiting list, or confirmed reservation. A waiting list reservation may become confirmed when a spot becomes available. Reservations can expire when the event starts or ends, depending on their current state.

3.2.2 Pregame Room Capacity State Machine

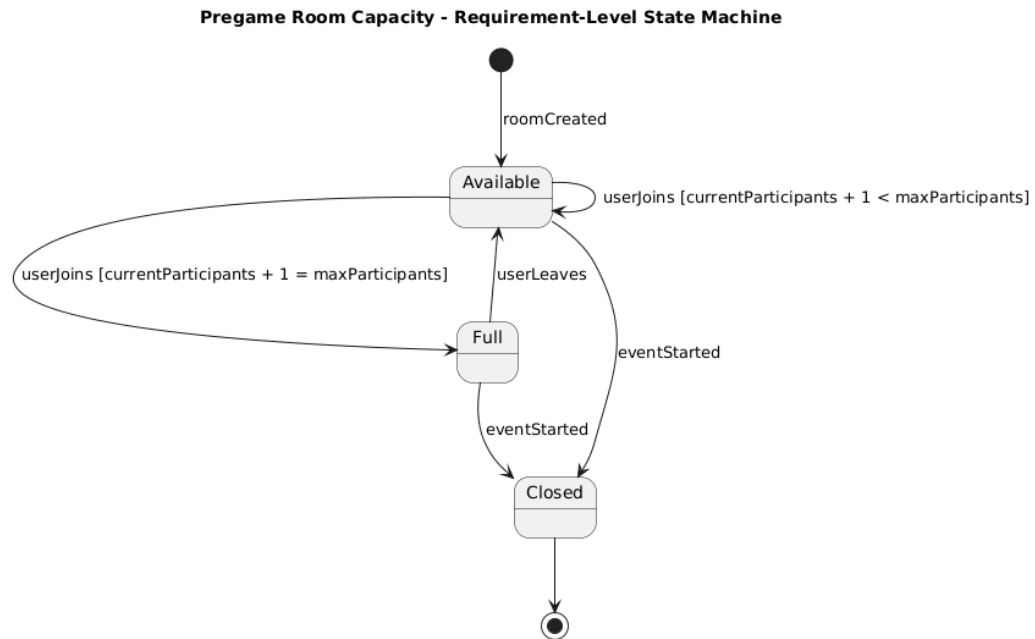


Figure 7: Pregame Room Capacity - Requirement-Level State Machine

A pregame room is available after creation. Users can join while the number of participants is lower than the maximum capacity. When the room reaches maximum capacity, it becomes full. If a participant leaves a full room, the room becomes available again. When the linked event starts, the pregame room is closed.